CHERR.IO



CONTRACTS OWNER GUIDE

Running the CHERR.IO smart contracts safely

VERSION 1.15 · OCTOBER 2026

DATA VALLIS D.O.O., SLOVENIA

CHERR.IO

CONTENTS

01	The short version	3
02	The contracts	4
03	Roles: who may do what	5
04	How fast a change happens	6
05	Every setting explained	7
06	Changing a setting, step by step	9
07	Example: a 1-hour vote window for testing	11
08	Other owner actions	12
09	If the console is not available	17
10	Checklist and troubleshooting	18
11	Keeping this guide current	20

ABOUT THIS GUIDE

For the owners of CHERR.IO. This guide explains what the CHERR.IO smart contracts let their owners change, what every setting means, which values are allowed, who may do what, and how long a change takes. It is written for the platform admin who holds the owner wallet (today David Tacer, Data Vallis d.o.o.). It follows the code in the public repository (`packages/contracts` , `apps/web/src/lib/contracts`) and decisions ADR-009, ADR-025, ADR-045, ADR-046 and ADR-050. If this guide and the code disagree, the code is right and this guide must be corrected.

01 THE SHORT VERSION

- Platform settings live in one contract, **PlatformConfig**. You change them on CHERR.IO under **Admin → Contracts**, in normal units: percent, minutes, hours, days, USDC.
- No setting changes at once. Every change goes through the **timelock**: you *schedule* it, wait, then *apply* it. On the test network (Amoy) the wait is **5 minutes**; on the main network it will be **48 hours**. Everyone can see a scheduled change on the blockchain before it happens. CHERR.IO lists every scheduled change — whoever scheduled it — on the public page **Scheduled contract changes** (`/en/contract-changes`), and the Emergency Pool page shows a notice while a change of where pool money goes is waiting (TASK-023d).
- A change applies only to **campaigns and Emergency Pool votes created after it is applied**. A running campaign keeps the values it started with, for its whole life.
- Your wallet signs everything. CHERR.IO's server holds no key and cannot change anything on its own.
- The contracts cannot be upgraded and cannot send money to an address of your choosing. The owner powers are limited to the settings and decisions in this guide.

WHAT YOU NEED

A computer with MetaMask (or another wallet CHERR.IO supports), the owner wallet imported in it, and a little POL in that wallet for network fees. On Amoy, test POL comes from a faucet. Since TASK-049 also an **authenticator app** on your phone (Google Authenticator or similar): every admin page asks for its code (§10).

02 THE CONTRACTS

CONTRACT	WHAT IT DOES	CHANGEABLE?
PlatformConfig	Holds every platform setting and the list of who has which role.	Settings: yes, through the timelock.
TimelockController	The owner of PlatformConfig. Executes a change only after the waiting time.	Its waiting time is fixed at deployment.
CampaignFactory	Creates one campaign contract per approved campaign.	No settings.
Campaign (one per campaign)	Holds the donations of one campaign: payouts, votes, refunds. Copies the settings when it is created.	Never.
EmergencyPool	Holds the Emergency Pool and its themed sub-pools; moves money to campaigns only by vote.	No settings.

Current addresses on the test network (dev, Polygon Amoy, chain 80002):

CONTRACT	ADDRESS
PlatformConfig	`0x4d2570ccB2a6653D62a002027C0d383FfB193A16`
TimelockController	`0x52ba2090E62c9155c04E7E5f28DB9Af00A6AAede`
CampaignFactory	`0xd5Ca76A8FC6E15C6cC3F3C691A2b6c70D9715a00`
EmergencyPool	`0xFa7Fd0253813E196d74575A8F93ABB91cd009517`

The main network addresses are added here when the contracts are deployed there. The source of truth is `packages/contracts/deployments/`.

03 ROLES: WHO MAY DO WHAT

ROLE	HELD BY	MAY DO	SPEED
Proposer (timelock)	The Timelock Safe	Schedule a settings change.	Change waits the timelock delay.
Executor (timelock)	The Timelock Safe	Apply a scheduled change once it is ready.	—
Canceller (timelock)	The Timelock Safe	Cancel a scheduled change before it is applied.	Immediate.
Admin (PlatformConfig)	The timelock itself	Every settings function and every role grant. Nobody calls these directly.	Only through the timelock.
Operator	The Operator Safe	Publish an approved campaign (CampaignFactory), set a campaign's payout mode, create Emergency Pool sub-pools, propose Emergency Pool allocations.	Immediate.
Guardian	The Guardian Safe	Freeze a campaign; decide a campaign or allocation under review (approve or reject).	Immediate, no timelock.

On Amoy a single test wallet, `0x4326…B5a7`, holds every role (ADR-025). On the main network there are **three different Gnosis Safe multisigs** with different signers — Operator, Guardian and Timelock — so that no one group can propose, decide and pay an Emergency Pool allocation alone (ADR-061, security review M-01). The deploy script refuses to give two roles to the same Safe.

THE GUARDIAN CANNOT TAKE MONEY

Freezing stops a campaign; resolving moves it to the next step or rejects it, after which donors get their money back or send it to the Emergency Pool, as each of them chose. No owner role can send funds anywhere else.

04 HOW FAST A CHANGE HAPPENS

ACTION	TEST NETWORK (AMOY)	MAIN NETWORK
Settings change (schedule → apply)	at least **5 minutes**	at least **48 hours**
Cancel a scheduled change	at once	at once
Guardian freeze or review decision	at once	at once
Publish a campaign, set payout mode	at once	at once

The waiting time is the timelock's *minimum delay*. It is shown on Admin → Contracts under "Waiting time before a change can be applied". On the main network it is fixed at 48 hours in the deployment script and cannot be shortened.

After a change is applied, **new** campaigns use it. A campaign created before keeps the old value until it ends. Example: if you shorten the vote window to 1 hour today, a campaign published yesterday still gives its donors the old window.

05 EVERY SETTING EXPLAINED

SETTING	WHAT IT MEANS	YOU ENTER	ALLOWED	DEFAULT
Platform fee	Share of each payout that goes to the CHERR.IO fee wallet. In a three-step campaign a third of the fee goes with each payment, so a rejected campaign pays no fee on the parts it never received (ADR-061).	percent, up to 2 decimals	0 – 5 %	1 %
Success threshold	Share of the target a campaign must raise by its deadline to succeed. Below it, donors get their money back.	percent	0.01 – 100 %	10 %
Vote window	How long donors can vote on a payout step after the fundraiser submits evidence.	number + minutes / hours / days	1 hour – 14 days	7 days
Quorum	Share of the donated amount that must take part in a vote. Below it, the vote goes to the Guardian.	percent	0.01 – 100 %	25 %
Approval	Share of the votes cast that must say yes for the next payout.	percent	more than 50 %, up to 100 %	51 %
Unclaimed refunds kept for	After this time, refunds nobody claimed can be moved to the Emergency Pool.	number + days	30 – 365 days	180 days
Minimum donation	The smallest donation a campaign accepts.	USDC, up to 6 decimals	more than 0, at most 1,000 USDC	1 USDC
Wait before a single payout	Time between the end of a campaign and its one-time payout, so the Guardian can step in.	number + minutes / hours / days	0 – 7 days	3 days
Fee wallet	Address that receives the platform fee.	address	any address except zero	set at deployment
Emergency Pool contract	Address of the Emergency Pool. Change only when a new pool contract is deployed.	address	any address except zero	set at deployment

“Default” is the value in the contract source (ADR-045). A contract deployed earlier keeps the values it was deployed with until you change them: the Amoy PlatformConfig was deployed with a 24-hour vote window and a 50 % quorum. Admin → Contracts always shows the live value under “Now”.

Votes in plain numbers. With a 25 % quorum and 51 % approval, a campaign that raised 1,000 USDC needs donors holding at least 250 USDC of it to vote, and more than half of the voted amount to say yes. Each donor's vote weighs as much as they donated. If nobody votes, or too few, the Guardian decides — silence is never counted as yes (ADR-045).

On-chain units. The contract stores percent as *basis points* (1 % = 100), times in *seconds* and USDC in *millionths* (1 USDC = 1,000,000). The console converts for you and shows the exact stored number in brackets, for example "25% (2500)".

06 CHANGING A SETTING, STEP BY STEP

1. **Open the console.** Log in to CHERR.IO with your admin account, open **Admin → Contracts**. The page shows the network, the contract addresses, the waiting time and every setting with its current value.
2. **Connect the owner wallet.** Connect MetaMask with the owner wallet. Under “Setup” the page lists your wallet and its roles; you need *can schedule changes* and *can apply changes*. If it says “no role on these contracts”, you are using the wrong wallet.
3. **Type the new values.** Change one or more fields. For times, pick the unit (minutes, hours, days). A field outside the allowed range turns red and says the lowest or highest allowed value. Under a changed field you see “Will be …”.
4. **Review.** Press **Review N changes**. A table shows each setting with “Now” and “New”, in words and as the stored number. Read it carefully.
5. **Schedule.** Press **Schedule the change** and confirm in MetaMask. The page says “Transaction sent. Waiting for the network to confirm it…” and then “The change is scheduled”. The change now waits in the timelock, visible to everyone. It appears under **Scheduled changes** with the time from which it can be applied. MetaMask only signs: every check before and after (your roles, the network fee, the confirmation) is read by CHERR.IO itself, so a slow MetaMask network connection does not stop the console.
6. **Wait.** 5 minutes on Amoy, 48 hours on the main network. You can close the page.
7. **Apply.** Come back. When the change says **Ready to apply**, press **Apply the change** and confirm in MetaMask. The new value appears under “Now”. Done.

To withdraw a scheduled change, press **Cancel** next to it and confirm in MetaMask. A cancelled change can never be applied; schedule a new one if needed.

Every step is recorded: who scheduled, applied or cancelled what and when, with the transaction (the console’s history and the admin audit log).

A CHANGE YOU DID NOT SCHEDULE HERE

The public page **Scheduled contract changes** shows every operation on the timelock, not only the ones made from this console; each says “Scheduled from: CHERR.IO admin console” or “Outside the admin console”. Changes that would move money (the Emergency Pool contract, the fee wallet) or roles are marked. When a change made outside the console is waiting, **Admin → Contracts** shows a yellow notice with the number of such changes and a link to the list. Every platform admin also gets an **email** a few minutes after any change is scheduled (subject “Contract change scheduled…”), saying when it can be applied, whether it moves money or roles, and whether it came from this console. The email comes even if you switched off campaign emails.

What to do: if it is yours (for example scheduled from the Safe app on the main network), nothing. If it is not, **cancel it before it can be applied**: the change's id is shown on the public page ("Change 0x…", the full id in its tooltip and in the address of the page, `#change-0x…`). Call `cancel(id)` on the timelock with a wallet that may cancel (the Safe app, or the block explorer's *Write contract* tab). A cancelled change can never be applied. Then find out how a proposer key was used without you.

MAIN NETWORK: THE SAFE

On the main network the owner is a multisig Safe, not MetaMask. When no connected wallet may schedule, the review offers **Download for Safe**: a file you import in the Safe app (Transaction Builder) to sign there. Applying works the same way after the 48 hours.

07 EXAMPLE: A 1-HOUR VOTE WINDOW FOR TESTING

On the test network we keep the vote window short so the whole donation → vote → payout flow can be tested in an afternoon (decision of 4 October 2026; the main network stays at 7 days).

1. Admin → Contracts, connect the owner wallet.
2. **Vote window**: type `1`, choose **hours**.
3. **Quorum**: type `25`.
4. **Review 2 changes** → check “1 day → 1 hour” and “50% → 25%” → **Schedule the change** → confirm.
5. After 5 minutes (the timelock waiting time on Amoy — it has nothing to do with the 1 hour you are setting): **Apply the change** under **Scheduled changes** → confirm. Nothing changes until you apply: before that, “Now” still shows 1 day and 50%.
6. Campaigns published from now on have a 1-hour vote. Campaigns published before keep 24 hours.

08 OTHER OWNER ACTIONS

These do not go through the timelock. The campaign actions are on the **admin campaign page** (TASK-033d, TASK-033f); **Admin → Chain actions** lists every campaign that waits for one.

ACTION	ROLE	WHERE TODAY
Publish an approved campaign	Operator	Admin → Campaigns → campaign → Publish (signed in your wallet).
Publish demo campaigns (test networks only)	Operator	Admin → Demo campaigns → **Publish all N demo campaigns** — up to 10 per round, one confirmation per campaign (ADR-052).
Set a campaign's payout mode (single or three milestones)	Operator	Admin → Chain actions → campaign → **Set the payout plan**. Only after the campaign succeeded, only once.
Decide a campaign under review	Guardian	Admin → Chain actions → campaign → **Approve — release the next payment** or **Reject — donors get the rest back**. A note is required.
Freeze a campaign	Guardian	Admin campaign page → **Freeze the campaign** (note and confirmation required).
Unfreeze a campaign	Guardian	Admin → Chain actions → campaign → **Unfreeze — continue where it stopped** (or reject).
Finish a campaign nobody finished (7 days after the deadline)	none — any wallet	Admin → Chain actions → "Finish — nobody did for 7 days" → **Finish the campaign** (ADR-050).
Count a vote nobody counted (7 days after the vote ended)	none — any wallet	Admin → Chain actions → "Count the votes — nobody did for 7 days" → **Count the votes**.
Move unclaimed refunds to the Emergency Pool (after the refund window)	none — any wallet	Admin → Chain actions → "Move unclaimed refunds to the Emergency Pool" → **Move to the Emergency Pool** (confirmation required).
Create an Emergency Pool sub-pool	Operator	Admin → **Emergency Pool** → **Create on the blockchain** next to the theme (TASK-046). One confirmation per theme; a sub-pool cannot be deleted.
Propose an Emergency Pool allocation	Operator	Admin → **Emergency Pool** → **Propose an allocation**: sub-pool, live campaign, amount, public reason → **Propose and sign** (TASK-014c). The amount is set aside at once; the contributors of that sub-pool vote.

Decide an Emergency Pool allocation under review	Guardian	Admin → **Chain actions** → **Emergency Pool allocations to decide** → **Send to the campaign** or **Return to the sub-pool** (TASK-014c-3). A note is required.
Grant or revoke a role	Admin (timelock)	Only through the timelock by hand (section 9). Ask the CTO session first.

CREATING THE EMERGENCY POOL SUB-POOLS

When a campaign fails, a donor may send their money to a themed sub-pool instead of taking a refund. CHERR.IO offers a theme only when it exists twice: as a row in the database (the five themes General 0, Medical emergencies 1, Natural disasters 2, Animals in danger 3, Climate 4 come with every deploy) and as a sub-pool on the blockchain. The general pool (0) is created with the contract; the other four are created once per network by the Operator.

1. Open **Admin → Emergency Pool**. The table shows each theme, its number, whether it is **on the blockchain** and its balance.
2. Connect the Operator wallet in the header (MetaMask; on Amoy 0x4326…B5a7).
3. Under **Create the missing sub-pools** click **Create on the blockchain** next to a theme and confirm in MetaMask. The Operator wallet pays the network fee (POL). CHERR.IO checks the call first: a theme that already exists or a wallet without the Operator role never reaches MetaMask, and the page says why.
4. "Created." appears. After about a minute the table shows **Yes** and the donate panel offers the theme. Repeat for the other themes.

What it does on the contract: `EmergencyPool.createSubPool(id)` marks the sub-pool as existing and emits `SubPoolCreated`. It moves no money. It cannot be undone; a donation to a sub-pool that does not exist goes to the general pool.

PROPOSING AN ALLOCATION FROM A SUB-POOL (TASK-014C)

Money in a sub-pool goes to a campaign only when the people who gave to that sub-pool vote yes. You start the vote.

1. Open **Admin → Emergency Pool** and connect the Operator wallet (MetaMask; on Amoy 0x4326… B5a7).
2. Under **Propose an allocation** choose the **Sub-pool** (its available balance is shown), the **Live campaign** (only campaigns live on the blockchain are offered), the **Amount** in USDC and a **Public reason** (up to 1,000 characters — it appears on the public Emergency Pool page exactly as written).
3. Press **Propose and sign** and confirm in MetaMask. CHERR.IO first saves the reason and checks the call: a wallet without the Operator role, a sub-pool without enough money, a campaign that is not live or ends before the vote would end, or a campaign that already got money from (or has an open proposal from) another sub-pool never reaches MetaMask, and the page says why.

4. "Proposed." appears. A few minutes later the vote shows on the public **Emergency Pool** page with the reason and its SHA-256 for anyone to check.

What it does on the contract: `EmergencyPool.proposeAllocation(poolId, campaign, amount, reasonHash)` sets the amount aside from the sub-pool, snapshots the vote window, quorum and approval share, and opens the vote (`AllocationProposed`). `reasonHash` is the SHA-256 of the reason text. Who may vote: everyone who gave to that sub-pool **before** this proposal, weighted by what they gave. A campaign is tied to the first sub-pool that actually **delivers** money to it; while a proposal to it is open, only that proposal's sub-pool may propose more. A rejected or returned proposal ties nothing (ADR-061). If that campaign later fails, the money comes back to the same sub-pool — through **Reclaim**, or automatically when the campaign's unclaimed refunds are swept.

The vote itself happens on the public Emergency Pool page (TASK-014c-2): contributors see their weight and press Vote yes / Vote no (`voteAllocation`); after the end **anyone** can press **Count the vote** (`closeAllocation`) — with enough turnout and yes votes the money goes to the campaign, otherwise back to the sub-pool; with too little turnout the allocation waits for the Guardian (next section).

DECIDING AN ALLOCATION UNDER REVIEW (TASK-014C-3)

An allocation goes to review when the vote ended with turnout under the quorum, or when nobody had given to that sub-pool before the proposal (so nobody could vote). The money stays set aside until the Guardian decides.

1. Open **Admin → Chain actions**. The section **Emergency Pool allocations to decide** lists each one: number, amount, sub-pool, campaign, why it is in review (turnout against the quorum, or "nobody had voting weight") and the public reason.
2. Connect the Guardian wallet (MetaMask; on Amoy 0x4326…B5a7). A wallet without the Guardian role cannot press the buttons, and the page says so.
3. Write a note (at least 10 characters — why you decide this way) and press **Send to the campaign** or **Return to the sub-pool**. The note is saved in the audit log first, then CHERR.IO checks the call and MetaMask opens.
4. "Sent." or "Returned." appears. A few minutes later the public Emergency Pool page shows the allocation as **Sent by CHERR.IO** or **Returned by CHERR.IO**. The note and the transaction are in **Admin → Audit log** (`pool.allocation_resolve.requested` / `.sent`).

What it does on the contract: `EmergencyPool.resolveAllocation(id, true)` sends the amount to the campaign (`AllocationResolved`, state RESOLVED_PASS); if the campaign takes less than the amount, the rest returns to the sub-pool; if the campaign no longer accepts money (it ended or was frozen), the allocation becomes **Not sent** and the whole amount returns to the sub-pool (`AllocationDeliveryFailed`). `resolveAllocation(id, false)` returns the amount to the sub-pool (RESOLVED_REJECT). It can be called once per allocation, only in review.

THE CAMPAIGN ACTIONS STEP BY STEP

1. Open **Admin → Chain actions**. Each row says what to do: “Set the payout plan”, “Decide the vote”, “Frozen — unfreeze or reject”, or one of the three steps after nobody acted (below). Click the campaign.
2. The section **On the blockchain — CHERR.IO actions** shows the state, the payout plan, what was raised and paid out, and for a vote the turnout against the campaign’s own quorum and the share of yes votes. Under it: the fundraiser’s evidence for each round. Private files are downloaded with **Download (recorded in the audit log)**.
3. Connect the wallet with the right role in the header (MetaMask). The page lists your connected wallets and their roles. On Amoy one wallet (0x4326…B5a7) is both Operator and Guardian.
4. Write the note and click the action. The note is saved first, then MetaMask opens. MetaMask only signs; CHERR.IO checks the call against the contract before it opens, so a call the contract would refuse never reaches MetaMask and the page names the reason.
5. After “Sent”, the transaction appears under **Notes and transactions** with your note. The state on the page changes when the indexer has seen the transaction (about a minute).

What each action does on the contract:

- **Payout plan.** `setPayoutMode(0)` = one payment, released 72 hours after the end. `setPayoutMode(1)` = three milestone payments; payments 2 and 3 need evidence and a donor vote. It cannot be changed later. An individual’s campaign is always milestones; the contract refuses one payment. The page suggests a plan: an organisation’s first campaign → one payment (supervised); later campaigns → one payment with a rating of 4.0 or more, milestones below or without a rating. The choice is yours.
- **Decide a vote.** A vote ends in review when turnout is under the quorum or there were no votes (ADR-045: no “silence = consent”). `resolve(true)` releases the next payment; `resolve(false)` rejects the campaign and donors can claim the rest back (or it goes to the Emergency Pool, by their choice).
- **Freeze.** `freeze()` stops everything — donations, votes, payments, refunds — in the states live, succeeded, paying, voting or in review. The contract takes no reason; your note is the record. Use it only for a serious problem (a fraud report, a wrong payout wallet).
- **Unfreeze.** There is no separate unfreeze function: `resolve(true)` on a frozen campaign returns it to the state it was in. An open vote gets back the time it was frozen, and so does a campaign that was still collecting donations: its deadline moves by the frozen time (ADR-061). `resolve(false)` rejects it.

WHEN NOBODY ACTED: CHERR.IO STEPS IN (ADR-050)

The people involved start almost every step themselves: the fundraiser or any visitor finishes a campaign after its deadline and counts a vote after it ends; donors claim their own refunds. If nobody does, the campaign appears in **Admin → Chain actions**, and on its page a box **CHERR.IO steps in** shows the one call that is due. There is no key on the server that could do it automatically — an admin sends it from their own wallet.

- **Finish the campaign** — `finalize()`, when the campaign is still running **7 days** after its deadline. The contract decides the result: success when it reached its threshold (10 % of the target), otherwise it fails and donors can get their money back.
- **Count the votes** — `closeVote()`, when a vote ended more than **7 days** ago and nobody counted it. The contract releases the next payment, rejects, or sends the vote to review, exactly as when the fundraiser does it.
- **Move to the Emergency Pool** — `sweepUnclaimed()`, when a failed or rejected campaign's refund window is over (the campaign's own "Unclaimed refunds kept for" value, 180 days by default, counted from the failure or the rejection). Everything still in the campaign goes to the Emergency Pool; donors can no longer claim a refund afterwards, so the page asks you to confirm.

The contract lets anyone call these three functions, so **no role is needed**: any connected wallet works and pays its own small network fee (POL). A note is optional. The 7 days are a rule of the CHERR.IO web app, not a contract setting. Before mainnet we decide whether a server key (or a Safe module) should do this automatically.

DEMO CAMPAIGNS ON THE TEST NETWORK (ADR-052)

On dev (and a local machine) **Admin → Demo campaigns** fills the platform with made-up organisations and campaigns for testing, in the same flow as production (ADR-053). It does not exist on uat or production — the server refuses there.

1. Enter how many new demo organisations (0–5; 0 adds campaigns to existing ones) and campaigns per organisation (1–5; at most 10 per batch), whether new campaigns are **approved** or **in review**, your **own test wallet** as payout wallet (payments and evidence of every demo organisation go to and come from it) and the duration: mixed (1, 3, 7, 14, 21, 30 days) or all **1 day**, the shortest the contract allows — for testing payouts quickly. **Create** makes each organisation with its own made-up member, who starts its campaigns; you are never a member, so the four-eyes rule applies as in production. Campaigns in review you approve in Admin → Campaigns. Covers are generated one by one on fal.ai with the model you choose under **Cover images**: **FLUX.2 [pro]** (default, cheaper, about 0.03 $ each) or **Nano Banana Pro** (about 0.15 $ each); the same choice sits next to **Generate missing covers**.
2. Connect the **Operator** wallet (on Amoy 0x4326…B5a7) and click **Publish all N demo campaigns**. For each campaign CHERR.IO prepares the call, and MetaMask asks you to confirm one `createCampaign` transaction — the same call as the single **Publish** button. The wallet's role is checked once per round. Rejecting a transaction in MetaMask stops the round; the rest stay waiting and the button publishes them next time.
3. Each published demo campaign is a real campaign contract on Amoy: donations, votes, refunds and payouts work as for any campaign. It carries a **Demo** tag and a notice on its page. Contracts cannot be deleted, so demo campaigns stay on dev.

09 IF THE CONSOLE IS NOT AVAILABLE

You can do exactly what the console does on Polygonscan, on the **timelock** contract, tab “Write Contract”, connected with the owner wallet:

1. `scheduleBatch` with: *targets* — the PlatformConfig address once per change; *values* — one `0` per change; *payloads* — the encoded setter call per change; *predecessor* — `0x` followed by 64 zeros; *salt* — any unused 32-byte value, e.g. `0x` + 63 zeros + `1`; *delay* — the waiting time in seconds (300 on Amoy).
2. Wait.
3. `executeBatch` with the same targets, values, payloads, predecessor and salt.

The payload of a setter is produced with Foundry: `cast calldata "setVoteWindow(uint32)" 3600`. The console is safer: it checks the allowed ranges, encodes for you and keeps the record. Changes made by hand do not appear in the console’s history.

10 CHECKLIST AND TROUBLESHOOTING

Before scheduling

- Right network in the wallet (Amoy for dev/uat, Polygon for production).
- The review table says what you mean, in words *and* in the stored number.
- You remember that only new campaigns are affected.

Admin sign-in: the authenticator code (ADR-056)

- **First time:** the first admin page you open says “Set up your authenticator”. Press “Show the QR code”, scan it with Google Authenticator (or type the key shown under it), type the 6-digit code and press “Confirm”. CHERR.IO then shows **ten recovery codes, only once**: write them down or save them away from your phone, then press “I saved them — continue”.
- **Afterwards:** when you open an admin page and the last code is older than **12 hours** (or you logged out), the page says “Confirm it is you”: type the current code from the app. Each code works once; if it is refused, wait for the next one.
- **Lost phone:** type one of the recovery codes instead of the 6-digit code (each works once), then set up the app again after a reset (below) if you have no phone.
- **Lost phone and no recovery codes:** on the server, in the web container of that environment: `node packages/db/dist/reset-admin-mfa.mjs <your wallet address>`. The next admin page asks you to set up the authenticator again. The reset is recorded in the audit log (`admin.mfa_reset`).
- Rotating `SESSION_SECRET` also resets every admin’s authenticator: everyone sets it up again (run the reset for each admin).

Finding your way and checking what happened (TASK-021)

- Every admin page has the same menu on top: Overview, Organisation applications, Campaigns, Organisations, Chain actions, Emergency Pool, Contracts, Audit log (and Demo campaigns on the test environments).
- **Admin → Audit log** lists every recorded action, newest first: who, what (`kyb.approve` , `contracts.change_scheduled` , `admin.mfa_enrolled` ...), on which record, and the details. Type part of an action in **Action**, choose **Subject type** or **Who** (“System” = done by CHERR.IO itself), click a name for only that person’s actions, or **history** for one record. Campaign, organisation and application pages have a link **Audit log of this record**.
- Check it after a reset or when something looks wrong: an `admin.mfa_enrolled` you did not do, or many `admin.mfa_failed` , means someone else tried to use an admin account — reset that admin’s authenticator and tell the CTO session. IP addresses are not shown in the page.

Messages you may see

MESSAGE	WHAT TO DO
“Your wallet is on another network.”	Switch the network in MetaMask and try again.
“This wallet may not schedule changes.”	Connect the owner wallet.
“The waiting time is not over yet.”	Wait until the change says “Ready to apply”.
“The wallet does not have enough POL for the network fee.”	Send POL to the owner wallet (on Amoy: faucet).
“You cancelled in your wallet. Nothing was sent.”	Nothing happened; try again if you meant to.
“The transaction was sent, but CHERR.IO could not record it.”	The change is on the blockchain; keep the transaction link and tell the CTO session so the record can be completed.
“Transaction sent, but its confirmation could not be read yet.”	The transaction was sent. Open the transaction link; when Polygonscan shows “Success”, reload the page — the change is under Scheduled changes. Do not schedule it again.
“Something went wrong. Try again.”	Nothing was confirmed by the console. If a transaction link is shown, check it first (as above). Otherwise try again; the browser console (F12) shows a line starting with `[contracts]` — send it to the CTO session.
“Not found on the blockchain”	The scheduling transaction failed or the change was cancelled; schedule again.

11 KEEPING THIS GUIDE CURRENT

The source of this guide is `docs/guides/owner/contracts-owner-guide.md` in the repository. Whenever the contracts, their roles, the deployment or Admin → Contracts change, the same pull request updates this guide, raises its version and adds a line to the change log in `docs/guides/owner/README.md`. The PDF is rebuilt from the source; never edit the PDF.



CHERR.IO

Running the CHERR.IO smart contracts safely

CHERR.IO · DATA VALLIS D.O.O., SLOVENIA · CONTRACTS OWNER GUIDE V1.15, OCTOBER 2026